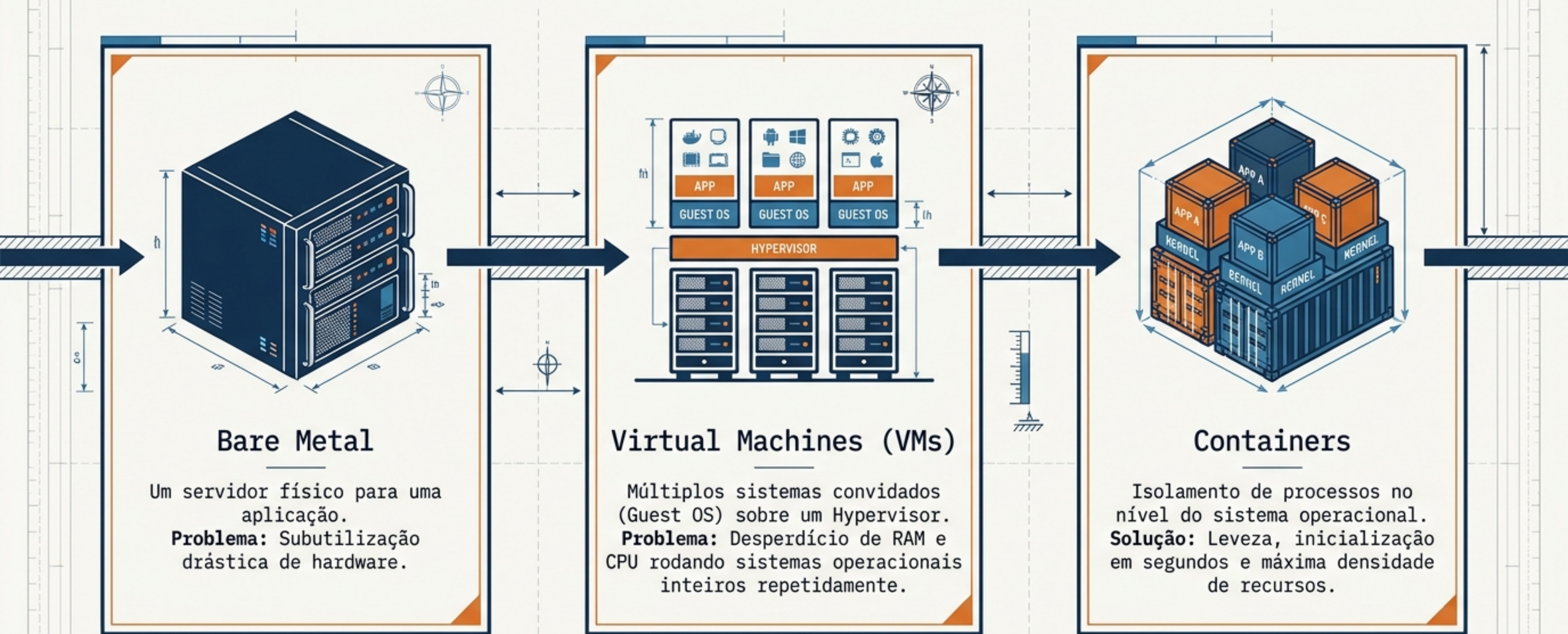


Arquitetura e Desenvolvimento: O Guia Definitivo de Docker

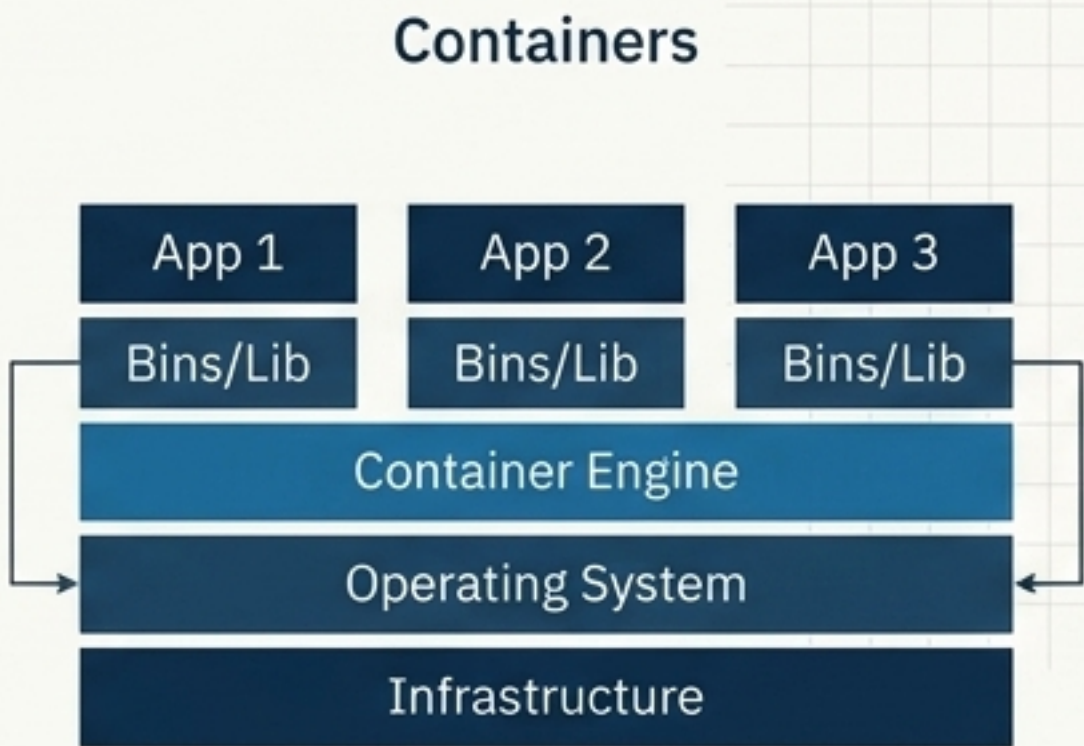
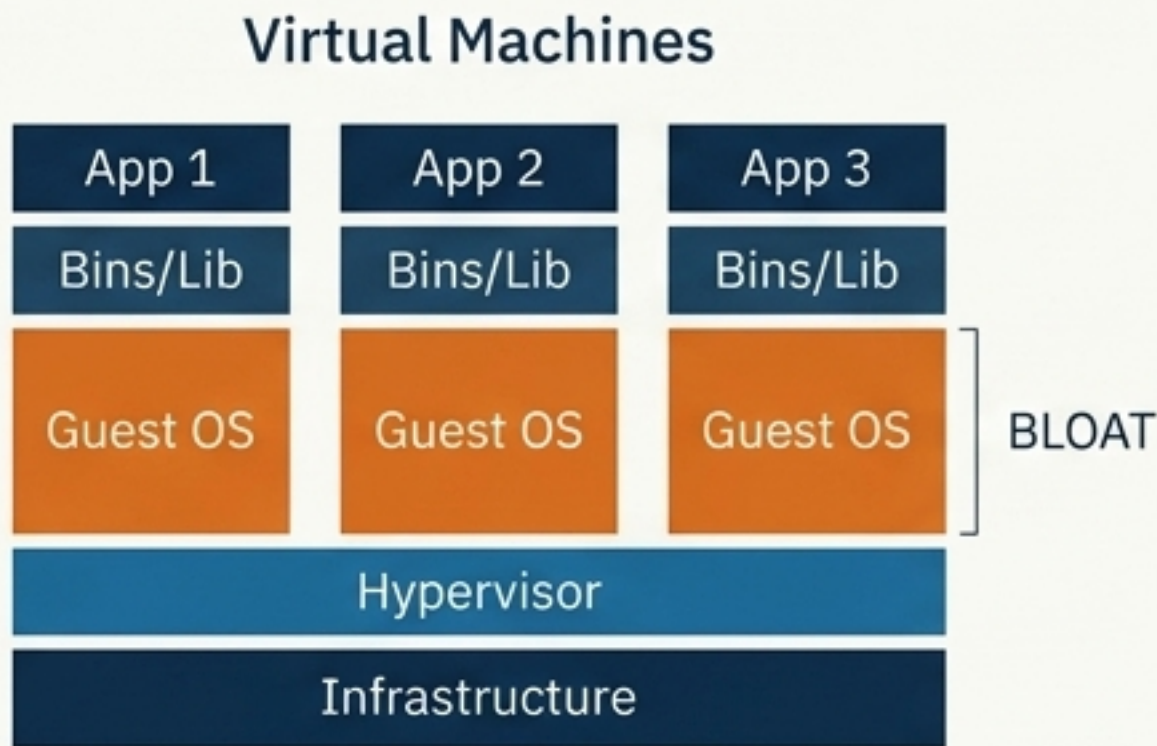
Dos fundamentos no Kernel
Linux à orquestração segura de
microsserviços.



A Evolução da Infraestrutura: Do Físico ao Lógico



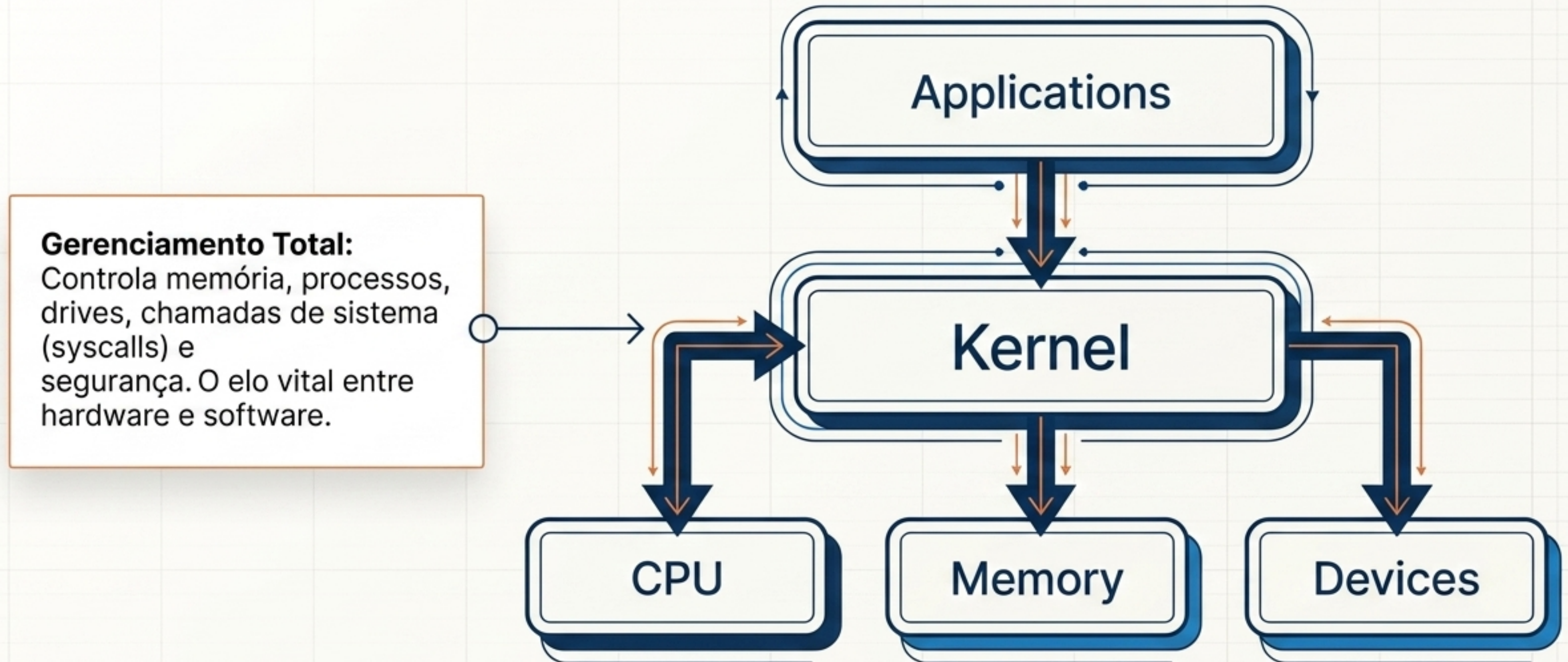
O Ponto de Virada: VMs vs. Containers



Característica	Virtual Machines	Containers
Arquitetura	Hypervisor + Guest OS	Container Engine + OS Compartilhado
Isolamento	Hardware Virtualizado	Processos (Nível de SO)
Peso Médio	Gigabytes (GBs)	Megabytes (MBs)
Tempo de Boot	Minutos	Segundos

O Motor Invisível: O Kernel Linux

O Docker empacota funcionalidades nativas de isolamento do Linux (criado em 2013 pela dotCloud) dispensando interfaces gráficas pesadas.



A Engenharia do Isolamento: Cgroups e Namespaces

Namespaces

(As paredes lógicas):

Garantem que um contêiner não veja o outro.
Criam redes, usuários e discos isolados.

Metáfora: Se um contêiner pega fogo ou cai no mar, o navio e os vizinhos permanecem intactos.



Cgroups (Os limites físicos):

Definem tetos rígidos de consumo. Controlam uso de CPU, Memória e priorização de processos vitais.

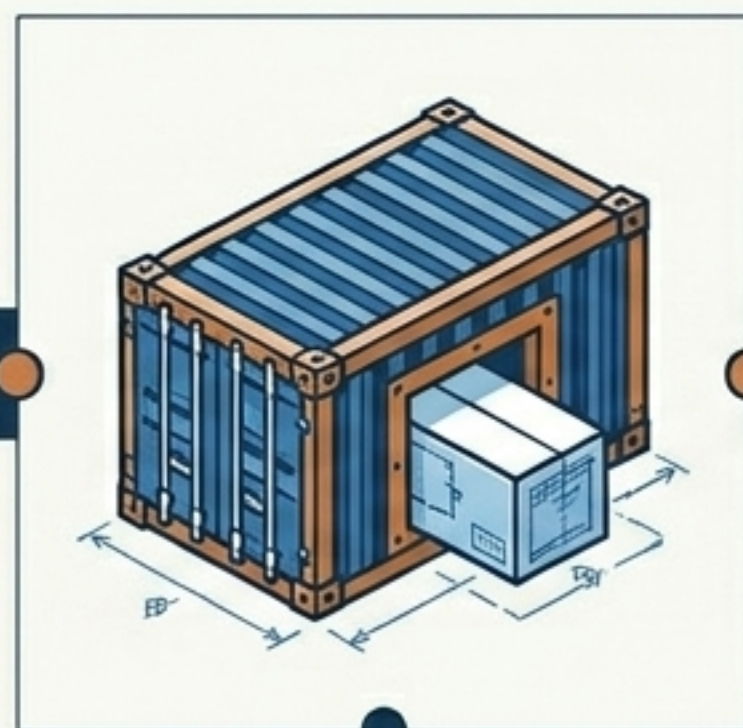
Ação: Impedem que um contêiner ruim sugue 100% dos recursos do host.

A Anatomia do Ecossistema: Da Receita à Execução



1. Dockerfile (A Receita)

O arquivo declarativo que empacota OS, dependências, variáveis e portas expostas.



2. Imagem & Tags (O Molde)

O artefato estático gerado. Tags garantem controle de versão e rollbacks seguros. Salvo em repositórios (Docker Hub - o GitHub das imagens).



3. Container (A Instância)

O processo isolado em execução consumindo a imagem. A aplicação viva.

Dissecando o Dockerfile (Node.js)

FROM: Imagem base oficial do Docker Hub com o ecossistema pré-instalado.

WORKDIR: Define o diretório isolado de trabalho interno.

```
FROM node:16.20.2
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .

EXPOSE 80
CMD [ "node", "app.js" ]
```

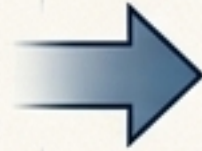
COPY / RUN: Instala as dependências via NPM de forma otimizada.

EXPOSE / CMD: Abre a porta 80 e engatilha o processo inicial da aplicação.

O Poder da Otimização: Multistage Builds



O Problema: Imagens base de desenvolvimento (node:18) são gigantes e cheias de ferramentas de compilação.



A Solução: Deploy hiper-rápido, menor custo de storage e menor superfície de ataque.

```
FROM node:18 as base
RUN apt-get update && \
    apt-get upgrade -y && \
    npm install -g newrelic && \
    newrelic save
COPY newrelic.js /app/newrelic
ENV NEWRELIC_LICENSEKEY \
    NEWRELIC_APPNAME
COPY . /app

FROM node:18 as builder
RUN npm build stage
COPY --from=base /app/newrelic /app
CMD newrelic start

FROM node:18-buster-slim as deploy
RUN
COPY --from=builder /app /app
CMD npm start
```

- Estágio 1 (as builder): Usa a imagem pesada apenas para instalar ferramentas e buildar o código.
- Estágio 2 (as deploy): Utiliza uma imagem minimalista (buster-slim), copiando apenas o artefato final compilado.

Painel de Controle: Monitorando Containers

> docker container ls -a

→ node-app docker container ls -a

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d6ba08789985	fiap:1.0	"docker-entrypoint.s..."	49 seconds ago	Exited (137) 8 seconds ago		inspiring_fermat
866423f5c290	fiap:2.0	"docker-entrypoint.s..."	About a minute ago	Up About a minute	80/tcp	pensive_mccarthy

→ node-app

CONTAINER ID

Hash alfanumérico único da instância.

IMAGE:

Qual versão/tag gerou este processo.

STATUS:

Saúde atual (Ex: Up (Rodando) ou Exited (137)- morto/parado).

PORTS:

O mapeamento vital (Porta do Host -> Porta do Container).

Governando Recursos na Prática (Cgroups em Ação)

A Ação: O Comando

```
docker container run -d  
--cpus=1 -m 512m fiap:2.0
```

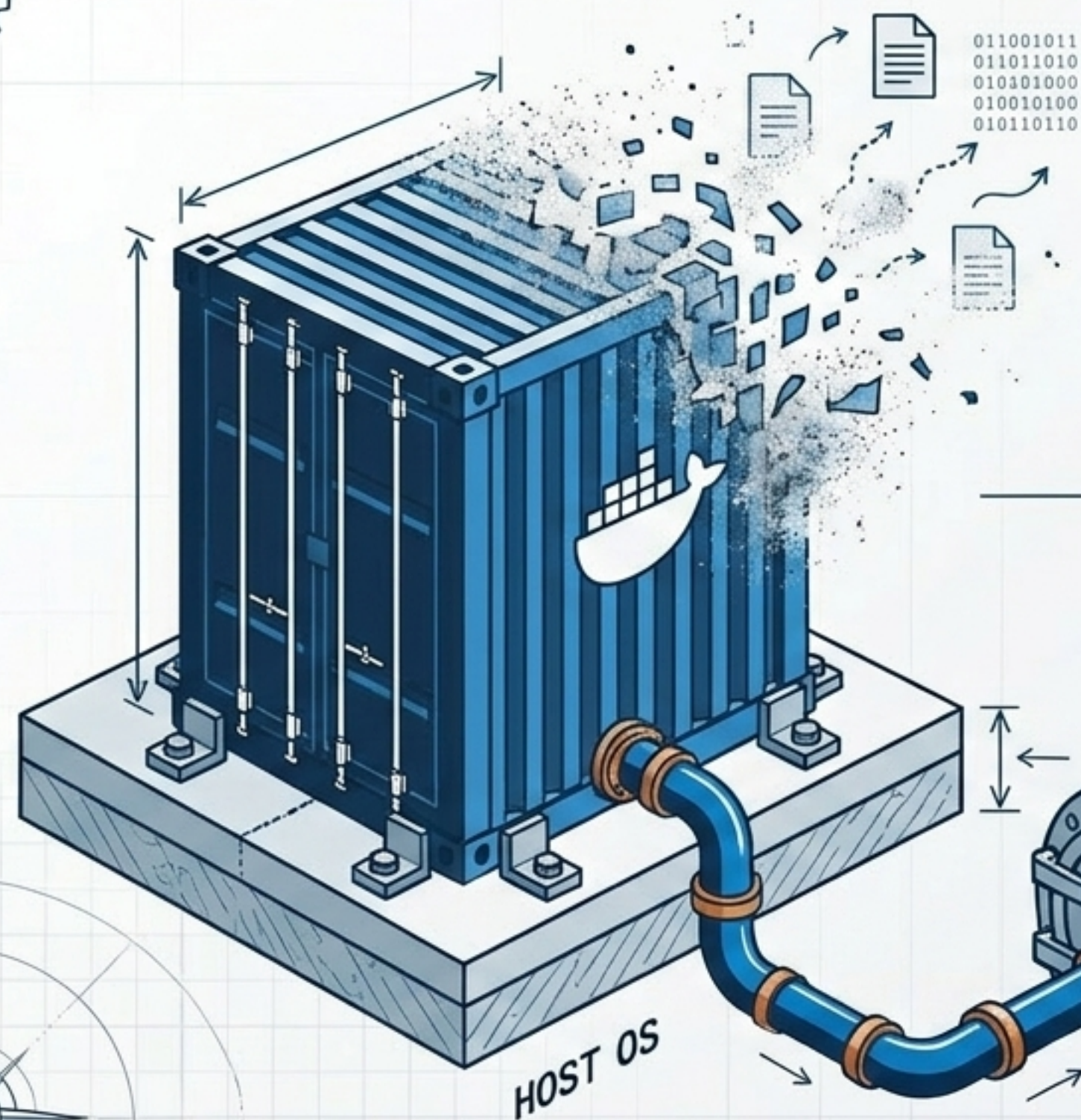
Limitamos rigidamente a instância a usar apenas 1 CPU Core e 512 MB de RAM, garantindo estabilidade no Host.

A Comprovação: A Auditoria

```
docker container inspect  
<ID> | grep -i mem  
"Memory": 536870912
```

O contêiner foi encapsulado exatamente nos limites estabelecidos de 512 MB.

O Problema da Efemeridade e a Solução dos Volumes



A Ameaça da Efemeridade: Por design, a camada de escrita do contêiner é volátil. Se ele ele morre, todos os dados internos evaporam.

A Solução Arquitetural: **Volumes**.

- Mapeiam um **diretório físico** no SO Host, persistindo dados independentemente do ciclo de vida do contêiner.
- **Data Containers:** Padrão onde contêineres dedicados compartilham volumes limpos, ideal para clusters de banco de dados (ex: MySQL).

Quebrando a Caixa Preta: Gestão de Logs

```
> docker container logs <container_id> --follow █
```

Contexto: →

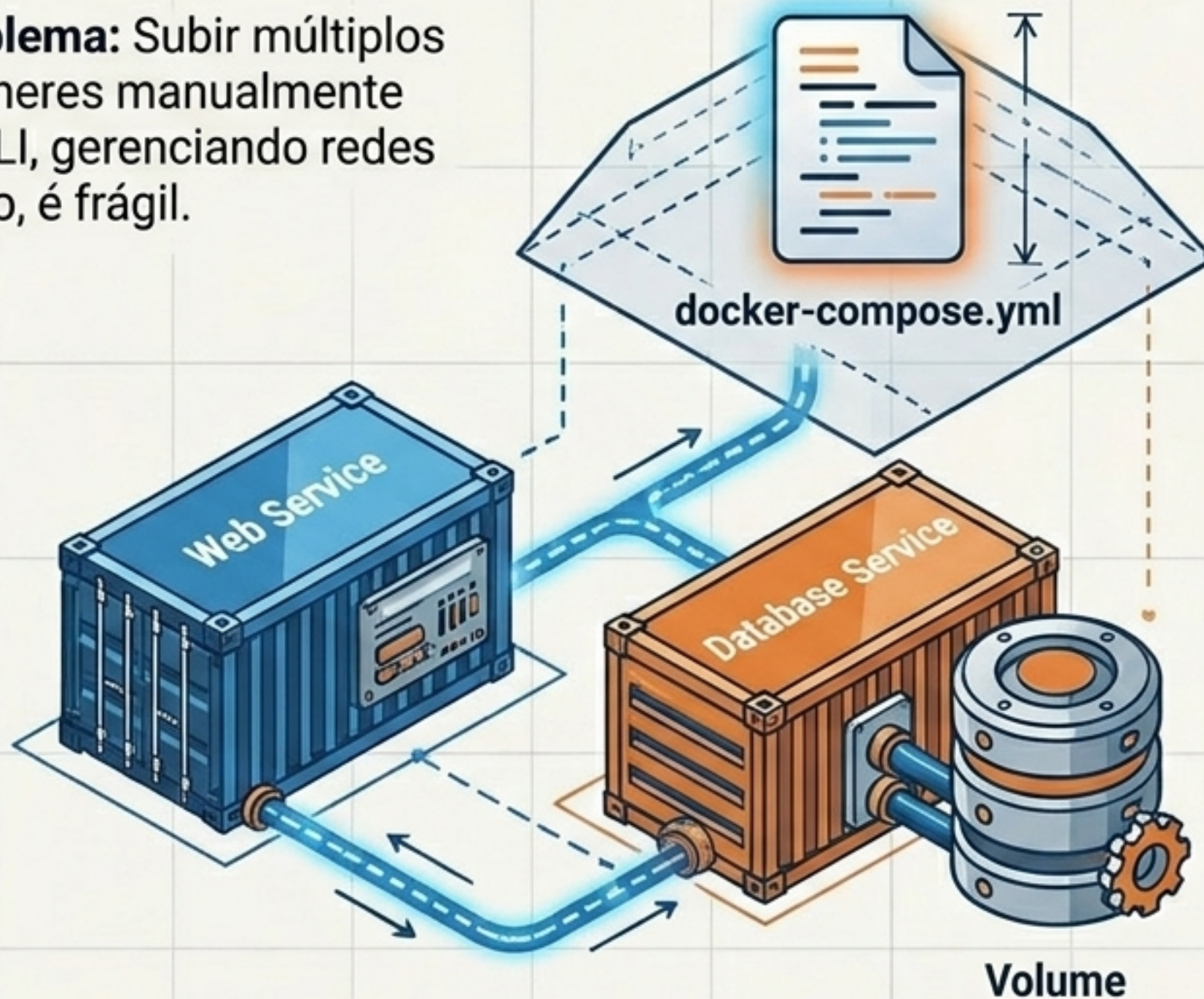
Rodar aplicações sem interface gráfica exige domínio sobre rastreamento de saídas. Tudo que vai para stdout/stderr é capturado pelo Docker.

Insight: >>>

O parâmetro **--follow** transforma o log estático em um stream ao vivo. Vital para debugar aplicações ou capturar as Death Messages de containers que sofrem crash repentino.

Subindo o Nível: Orquestração com Docker Compose

O Problema: Subir múltiplos contêineres manualmente pelo CLI, gerenciando redes na mão, é frágil.



Os 3 Pilares do docker-compose.yml:



1. Version:

Definam: Define a sintaxe e funcionalidades suportadas do arquivo.



2. Services: As aplicações (ex: app Node.js + Banco MySQL) declaradas com suas imagens.



3. Networks & Volumes:

Cria redes virtuais fechadas e discos externos persistentes automaticamente.

Shift-Left Security: Blindando sua Imagem com Trivy



O Desafio:

O Docker Hub facilita o uso de imagens base, mas elas podem trazer vulnerabilidades (CVEs) latentes instaladas nas dependências internas.

A Solução de Segurança: Trivy Scanner.

- Ferramenta Open Source vital.
- Atua localmente varrendo camadas da imagem em busca de brechas conhecidas, dependências defasadas ou secrets vazados antes de colocar o código em produção (Shift-Left).

O Mapa Geral da Arquitetura de Contêineres

Operations Board



Resultado Final: Uma arquitetura resiliente, leve, imutável e pronta para escalar na nuvem.